CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

CHANDIGARH UNIVERSITY

Gharuan, Mohali, Punjab

MINI PROJECT REPORT

Netflix Movies and TV Shows Analysis

Object Oriented Exploratory Data Analysis of Genres, Ratings and Release Years using Python Classes, Pandas, NumPy and Matplotlib

Subject Name	Object Oriented Programming (OOPS)
Submitted By	Arun Saini
UID	25bcd10196
Class	25BCD-3(A)
Submitted To	Mr. Abhinay Sood

Declaration and Acknowledgement

Declaration

I, **Arun Saini** (UID 25bcd10196, Class 25BCD-3(A)), hereby declare that this mini project report titled **"Netflix Movies and TV Shows Analysis"** is submitted as part of the Object Oriented Programming (OOPS) course. The class design, the Python code and the written report are prepared by me for learning purposes. The dataset used belongs to its original author and is cited in the references.

The project applies the main ideas of object oriented programming (classes, objects, encapsulation, abstraction, inheritance, polymorphism and composition) to a real data analysis problem.

Acknowledgement

I would like to thank my faculty guide **Mr. Abhinay Sood** for the guidance and the clear explanation of object oriented concepts throughout the course. His lectures helped me understand how a larger program can be divided into small, reusable classes.

I also thank **Chandigarh University** for providing the learning environment and resources needed to complete this work, and Shivam Bansal for making the Netflix dataset publicly available on Kaggle.

Arun Saini
UID: 25bcd10196
Class: 25BCD-3(A)

Table of Contents

1. Introduction 4
2. Objectives, Problem Statement and Scope 5
3. Unique Special Point of this Project 6
    3.1 Self-registering Analyzer Architecture 6
    3.2 Adding a New Analysis in 10 Lines 7
4. Tools and Libraries Used 8
5. OOP Concepts Used 9
6. Dataset Description 10
7. System Design 11
    7.1 Class Diagram 11
    7.2 Data Flow and Design Principles 12
8. Implementation (Class by Class) 13
    8.1 DataLoader and DataCleaner 13
    8.2 BaseAnalyzer (abstract class) 14
    8.3 Concrete Analyzer Classes 15
    8.4 Visualizer and NetflixPipeline 17
9. Testing 18
10. Results and Analysis 19
    10.1 Data Cleaning 19
    10.2 Movies vs TV Shows and Genres 20
    10.3 Ratings 21
    10.4 Release Years 22
    10.5 Duration, Countries and Growth 23
11. Conclusion, Limitations and Future Scope 24
12. References 25

1. Introduction

Netflix is one of the largest streaming platforms in the world. It offers thousands of movies and TV shows from many countries, and every title carries information such as its type, genres, audience rating, release year, country and duration. When this information is studied with the help of a program, it shows what kind of content the platform focuses on and how that focus has changed over time.

Most small data analysis programs are written as one long script, where loading, cleaning, calculating and plotting are mixed together. Such a script works, but it is hard to read, hard to test and hard to extend. In this mini project the same analysis is built in an **object oriented way**. Each job (reading the file, cleaning the data, analysing genres, analysing ratings, drawing charts) is given to its own class, and the classes work together through a small pipeline.

The analysis looks at three main aspects of the Netflix catalogue: **genres**, **ratings** and **release years**. Two extra aspects, movie duration and country of production, are added to show how easily a new analyzer class can be plugged in.

Why object oriented programming for data analysis?

- **Reuse:** the same cleaner and visualizer classes can serve any similar dataset.
- **Organisation:** every class has one clear responsibility, so the code is easy to read and explain.
- **Safety:** private attributes and properties protect important values, and a custom exception gives clear error messages.
- **Extension:** a new analysis is added by writing a new subclass, without changing the existing code.
- **Testing:** each class can be tested alone on a small sample of data.

2. Objectives, Problem Statement and Scope

2.1 Problem statement

To design and implement an object oriented Python program that loads the Netflix Movies and TV Shows dataset, cleans it, analyses genres, ratings and release years, and presents the results as readable text and charts, in such a way that new analyses can be added with minimum change to the existing code.

2.2 Objectives

- To model the analysis problem with classes and objects instead of a single script.
- To use an abstract base class so that every analyzer follows the same structure.
- To apply inheritance and polymorphism: six analyzers share one interface.
- To use encapsulation for the file path and for the list of cleaning steps.
- To clean the data by handling missing values and converting columns to suitable types.
- To find the most common genres, the rating distribution and the growth by release year.
- To use NumPy for statistics such as mean, median, standard deviation and percentiles.
- To present the findings through clear charts and short plain English insights.

2.3 Scope

The project works on the file *netflix_titles.csv* (7,787 titles, 12 columns). It covers descriptive analysis only: counting, grouping, cross tabulation and basic statistics. It does not predict anything and does not use machine learning. Cast and director columns are not analysed in this version.

2.4 Deliverables

Deliverable	Description
netflix_oop.py	Complete object oriented source code (8 classes plus one exception)
Charts	Bar charts, decade chart, genre by decade heatmap, class diagram
Report	This 25 page report with design, code, testing and results

3. Unique Special Point of this Project

3.1 Self-registering Analyzer Architecture

The special feature of this project is a **plug-in style design**. Normally, when a new analysis is needed, the programmer must edit the main program and add one more function call. In this project that step is not needed. All analyzers inherit from the abstract class `BaseAnalyzer`, and the class registers itself in a dictionary the moment it is defined, using Python's `__init_subclass__` hook.

The pipeline simply reads this dictionary and runs every registered analyzer. Because every analyzer must implement the same two abstract methods, `analyze()` and `insights()`, the pipeline can treat them all in the same way (polymorphism).

```
class BaseAnalyzer(ABC):
    registry = {}

    def __init_subclass__(cls, **kwargs):          # runs when a subclass is defined
        super().__init_subclass__(**kwargs)
        BaseAnalyzer.registry[cls.__name__] = cls   # automatic registration

class NetflixPipeline:
    def run_all(self):
        self.analyzers = [cls(self.df).run()        # no analyzer is named here
                          for cls in BaseAnalyzer.registry.values()]
        return self
```

What makes it special

- **Open for extension, closed for modification:** the pipeline code never changes when a new analysis is added (the Open/Closed principle of good design).
- **Built-in insights:** each analyzer does not only calculate numbers, it also writes plain English sentences through `insights()`, so the final report is generated automatically.
- **Method chaining:** calls such as `NetflixPipeline(path).prepare().run_all()` read like a sentence.
- **Audit trail:** the cleaner keeps a log of every cleaning step it performed.
- **Safe failure:** a custom `DatasetError` reports a missing file or missing columns in clear words.

3.2 Adding a New Analysis in 10 Lines

To prove that the design really is extensible, here is a complete new analyzer that counts how many titles were added to Netflix in each year. Nothing else in the project is edited. Once this class is written, the pipeline finds it and includes it in the report.

```
class YearAddedAnalyzer(BaseAnalyzer):
    title = "Titles added per year"

    def analyze(self):
        self.result = self._df["year_added"].dropna().astype(int).value_counts().sort_index()

    def insights(self):
        best = self.result.idxmax()
        return [f"Most titles were added in {best} ({self.result.max()} titles)"]
```

Comparison with a normal script

Point	Normal single script	This OOP project
Adding a new analysis	Edit the main code and add new calls	Write one subclass; it registers itself
Testing one part	Run the whole script	Create one object and call run()
Reading the code	One long file of mixed steps	Small classes, one job each
Sharing data safely	Global variables	Attributes inside objects
Error handling	Program crashes with a long traceback	Custom DatasetError with a clear message
Final report	Written by hand after the run	Generated from insights() of each analyzer

Summary of the unique points

No.	Unique point	OOP idea behind it
1	Analyzers register themselves automatically	Inheritance, __init_subclass__
2	Pipeline runs any analyzer without knowing its name	Polymorphism
3	Every analyzer writes its own plain English insights	Abstraction
4	Chained calls and a cleaning log	Method chaining, encapsulation
5	Clear errors through a custom exception	Exception handling

4. Tools and Libraries Used

Tool / Library	Purpose in this project
Python 3	Programming language; classes, inheritance, abc module
abc (standard library)	Abstract base class and abstract methods
Pandas	Reading the CSV file, cleaning, grouping, exploding genres
NumPy	Mean, median, standard deviation and percentiles of duration and year
Matplotlib	Drawing and saving charts (used inside the Visualizer class)
Seaborn	Heatmap and styled charts in the report
VS Code / Jupyter	Environment used to write and run the code

System requirements

Item	Requirement
Python version	3.8 or higher
Packages	pandas, numpy, matplotlib, seaborn
Install command	pip install pandas numpy matplotlib seaborn
Input file	netflix_titles.csv in the same folder as netflix_oop.py
Run command	python netflix_oop.py

Project files

```
project/
 |-- netflix_oop.py        # all classes (loader, cleaner, analyzers, visualizer, pipeline)
 |-- netflix_titles.csv    # dataset (7,787 rows, 12 columns)
 |-- charts/               # images saved by the Visualizer class
 `-- report.pdf            # this report
```

All the logic is kept in a single module so that the project is easy to submit and run. In a larger project every class could be moved into its own file without changing how the classes work together.

5. OOP Concepts Used

The table below lists each object oriented concept and shows exactly where it appears in the project code.

Concept	Where it is used	Why it helps
Class and object	DataLoader, DataCleaner, Visualizer, NetflixPipeline and six analyzers	Each part of the problem becomes a clear unit
Encapsulation	DataLoader.__path is private and read through the path property; DataCleaner keeps its own log	Important values cannot be changed by mistake
Abstraction	BaseAnalyzer with abstract methods analyze() and insights()	Defines what an analyzer must do, not how
Inheritance	TypeAnalyzer, GenreAnalyzer, RatingAnalyzer, ReleaseYearAnalyzer, DurationAnalyzer, CountryAnalyzer extend BaseAnalyzer	Common code (run, __str__) is written once
Polymorphism	NetflixPipeline.run_all() calls run() on every analyzer in the same way	Pipeline works with any analyzer type
Composition (has-a)	NetflixPipeline owns a DataLoader and a list of analyzers	Builds big behaviour from small objects
Constructors	__init__ in every class; GenreAnalyzer adds top_n using super().__init__	Objects start in a valid state
Class variables	BaseAnalyzer.registry, DataCleaner.RATING_GROUPS, DataLoader.REQUIRED	Data shared by all objects of a class
Properties	DataLoader.path	Read only access to a private value
Dunder methods	__repr__, __str__, __len__, __init_subclass__	Objects print and behave naturally
Method chaining	DataCleaner steps return self; pipeline prepare().run_all()	Short and readable code
Exception handling	DatasetError raised by DataLoader.load()	Clear message instead of a crash

6. Dataset Description

The dataset used is the **Netflix Movies and TV Shows** dataset by Shivam Bansal (Kaggle), collected from the Flixable search engine and listing titles available on Netflix up to early 2021. The file *netflix_titles.csv* contains **7,787 rows** and **12 columns**. Each row is one title.

Column	Description
show_id	Unique ID of the title
type	Movie or TV Show
title	Name of the title
director	Director(s) of the title
cast	Main actors
country	Country or countries of production
date_added	Date on which it was added to Netflix
release_year	Year in which the title was originally released
rating	Audience maturity rating (TV-MA, PG-13, etc.)
duration	Minutes for movies, number of seasons for TV shows
listed_in	Genres / categories (comma separated)
description	Short summary of the title

How the dataset is used by the classes

Column(s)	Used by class
type	TypeAnalyzer, GenreAnalyzer.by_type(), DurationAnalyzer
listed_in	DataCleaner (split into a genres list), GenreAnalyzer
rating	DataCleaner (audience group), RatingAnalyzer
release_year	DataCleaner (decade), ReleaseYearAnalyzer
duration	DurationAnalyzer
country	DataCleaner (fill missing), CountryAnalyzer
date_added	DataCleaner (year_added)

7. System Design

7.1 Class Diagram

The diagram shows all classes of the project. The six analyzers inherit from the abstract class BaseAnalyzer. NetflixPipeline is the controller: it uses DataLoader and DataCleaner to prepare the data and creates the analyzers found in the registry.

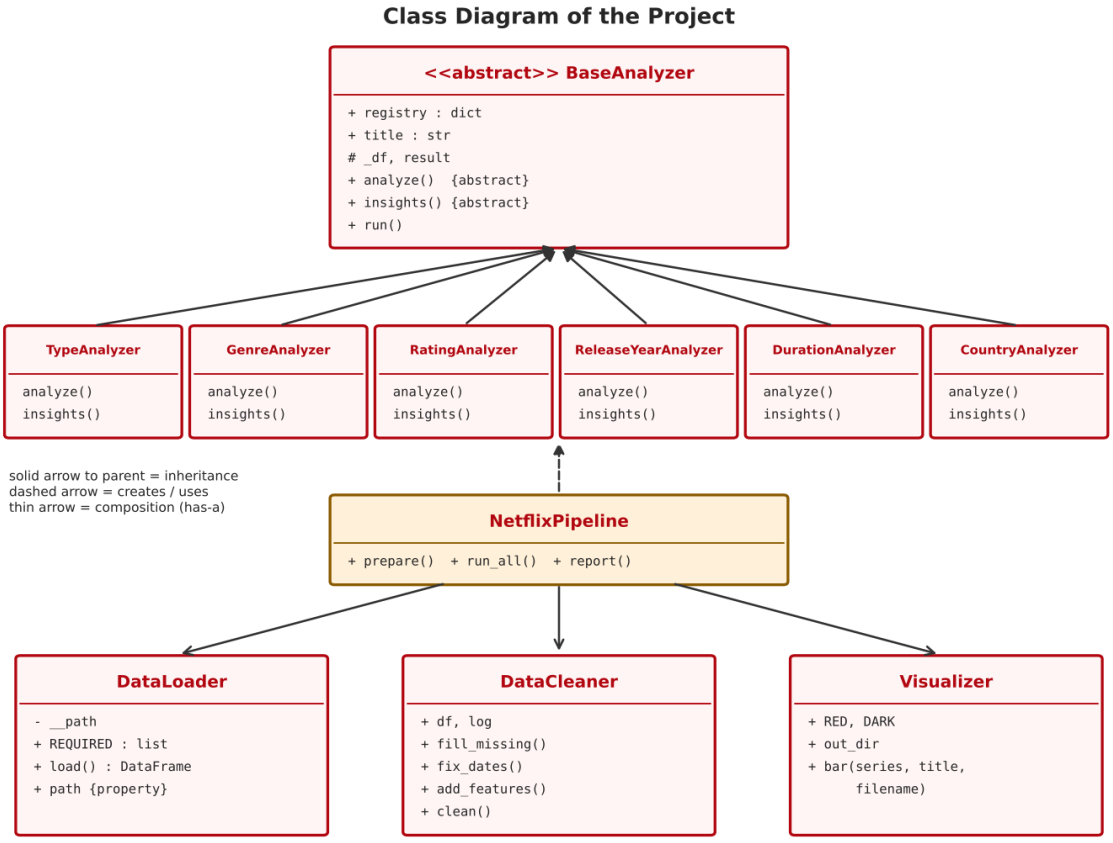


Figure 1: Class diagram of the Netflix analysis project

7.2 Data Flow and Design Principles

The data moves through the program in six steps. The file is read by DataLoader, cleaned by DataCleaner, and then every analyzer found in the registry is created and run. The results end as a text report and as charts.

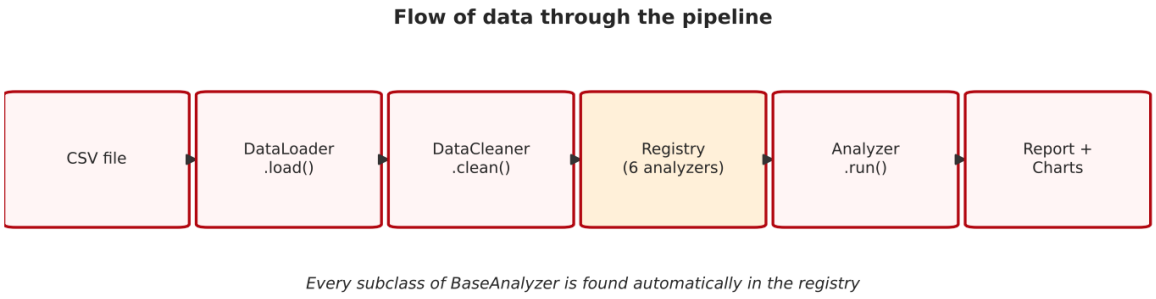


Figure 2: Flow of data through the pipeline

Responsibility of each class

Class	Single responsibility	Main methods
DataLoader	Read the CSV and check columns	load(), path
DataCleaner	Fix missing values, dates and create new columns	fill_missing(), fix_dates(), add_features(), clean()
BaseAnalyzer	Define the common shape of every analysis	analyze(), insights(), run()
Six analyzers	One analysis each	analyze(), insights()
Visualizer	Draw and save charts	bar()
NetflixPipeline	Connect all objects together	prepare(), run_all(), report()

Design principles followed

- **Single responsibility:** every class does one job only.
- **Open/Closed:** new analyzers are added without editing the pipeline.
- **Liskov substitution:** any analyzer can be used where BaseAnalyzer is expected.
- **Composition over inheritance for control:** the pipeline has objects instead of inheriting from them.

8. Implementation (Class by Class)

8.1 DataLoader and DataCleaner

DataLoader keeps the file path private and checks that all needed columns are present. **DataCleaner** works on a copy of the data and returns `self` from each step so that the steps can be chained.

```
class DatasetError(Exception):
    """Raised when the dataset is missing or has wrong columns."""


# 2. DataLoader - responsible only for reading data
class DataLoader:
    REQUIRED = ["type", "title", "country", "date_added",
                "release_year", "rating", "duration", "listed_in"]

    def __init__(self, path):
        self.__path = path            # private attribute (encapsulation)

    @property
    def path(self):
        return self.__path

    def load(self):
        try:
            df = pd.read_csv(self.__path)
        except FileNotFoundError:
            raise DatasetError(f"File not found: {self.__path}")
        missing = [c for c in self.REQUIRED if c not in df.columns]
        if missing:
            raise DatasetError(f"Missing columns: {missing}")
        return df

    def __repr__(self):
        return f"DataLoader(path='{self.__path}')"


# 3. DataCleaner - cleaning and feature creation
class DataCleaner:
    RATING_GROUPS = {
        "TV-MA": "Adults", "R": "Adults", "NC-17": "Adults",
        "TV-14": "Teens", "PG-13": "Teens",
        "TV-PG": "Older Kids", "PG": "Older Kids",
        "TV-Y7": "Kids", "TV-Y7-FV": "Kids", "TV-Y": "Kids",
        "TV-G": "Kids", "G": "Kids",
    }

    def __init__(self, df):
        self.df = df.copy()
        self.log = []                 # keeps a record of every step

    def fill_missing(self):
        for col in ("rating", "country"):
            n = int(self.df[col].isna().sum())
            self.df[col] = self.df[col].fillna("Unknown")
            self.log.append(f"{col}: {n} missing values filled with 'Unknown'")
        return self                   # returning self allows method chaining

    def fix_dates(self):
        self.df["date_added"] = pd.to_datetime(
            self.df["date_added"].astype(str).str.strip(), errors="coerce")
        self.df["year_added"] = self.df["date_added"].dt.year
        self.log.append("date_added converted to datetime")
        return self

    def add_features(self):
        self.df["decade"] = self.df["release_year"] // 10 * 10
        self.df["genres"] = self.df["listed_in"].str.split(", ")
        self.df["audience"] = (self.df["rating"].map(self.RATING_GROUPS)
                               .fillna("Other/Unrated"))
        self.log.append("decade, genres list and audience group created")
        return self

    def clean(self):
        return self.fill_missing().fix_dates().add_features().df
```

8.2 BaseAnalyzer (abstract class)

`BaseAnalyzer` inherits from `ABC`, so it cannot be used to create objects directly. A subclass that forgets to write `analyze()` or `insights()` cannot be created at all, which protects the design. The `run()` method and `__str__()` are written once here and inherited by every analyzer.

```
class BaseAnalyzer(ABC):
    """Every analysis is a subclass. Subclasses register themselves
    automatically, so a new analysis needs no change in the pipeline."""

    registry = {}
    title = "Base analysis"

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        BaseAnalyzer.registry[cls.__name__] = cls

    def __init__(self, df):
        self._df = df
        self.result = None

    @abstractmethod
    def analyze(self):
        """Compute and store the result."""

    @abstractmethod
    def insights(self):
        """Return a list of plain English sentences."""

    def run(self):
        self.analyze()
        return self

    def __str__(self):
        lines = [f"== {self.title} =="] + [f" - {s}" for s in self.insights()]
        return "\n".join(lines)
```

How the registry works

- When Python reads a line such as `class GenreAnalyzer(BaseAnalyzer)`, it automatically calls `BaseAnalyzer.__init_subclass__`.
- That method stores the new class in the dictionary `registry` under its name.
- `NetflixPipeline.run_all()` loops over this dictionary and builds one object per class.
- Result: the six analyzers of this project are registered without a single extra line.

8.3 Concrete Analyzer Classes

Each analyzer extends `BaseAnalyzer` and fills in the two abstract methods. `GenreAnalyzer` also shows how a child class adds its own attribute (`top_n`) by calling `super().__init__`, and how it can offer extra methods such as `by_type()`.

```
class TypeAnalyzer(BaseAnalyzer):
    title = "Movies vs TV Shows"

    def analyze(self):
        self.result = self._df["type"].value_counts()

    def insights(self):
        total = int(self.result.sum())
        return [f"{k}: {v} titles ({v / total * 100:.1f}%)"
                for k, v in self.result.items()]


class GenreAnalyzer(BaseAnalyzer):
    title = "Genre analysis"

    def __init__(self, df, top_n=10):
        super().__init__(df)
        self.top_n = top_n

    def analyze(self):
        exploded = self._df.explode("genres")
        self.result = exploded["genres"].value_counts()

    def by_type(self, content_type):
        sub = self._df[self._df["type"] == content_type].explode("genres")
        return sub["genres"].value_counts().head(self.top_n)

    def insights(self):
        top = self.result.head(3)
        names = ", ".join(f"{k} ({v})" for k, v in top.items())
        return [f"{len(self.result)} unique genres found",
                f"Top three genres: {names}"]


class RatingAnalyzer(BaseAnalyzer):
    title = "Rating analysis"

    def analyze(self):
        self.result = self._df["rating"].value_counts()

    def audience_share(self):
        return (self._df["audience"].value_counts(normalize=True) * 100).round(1)

    def insights(self):
        total = int(self.result.sum())
        first, second = self.result.index[0], self.result.index[1]
        share = (self.result.iloc[0] + self.result.iloc[1]) / total * 100
        return [f"Most common rating: {first} ({self.result.iloc[0]} titles)",
                f"{first} and {second} together cover {share:.1f}% of titles"]
```

RatingAnalyzer.audience_share() converts ratings into the audience groups Adults, Teens, Older Kids and Kids (created earlier by DataCleaner) and returns percentages.

```
class ReleaseYearAnalyzer(BaseAnalyzer):
    title = "Release year analysis"

    def analyze(self):
        ry = self._df["release_year"].values
        counts = self._df["release_year"].value_counts().sort_index()
        self.result = {
            "mean": float(np.mean(ry)), "median": float(np.median(ry)),
            "min": int(ry.min()), "max": int(ry.max()),
            "peak_year": int(counts.idxmax()), "peak_count": int(counts.max()),
            "share_since_2015": float((ry >= 2015).mean() * 100),
        }

    def insights(self):
        r = self.result
        return [f"Titles released between {r['min']} and {r['max']}",
                f"Mean year {r['mean']:.1f}, median year {r['median']:.0f}",
                f"Peak year {r['peak_year']} with {r['peak_count']} titles",
                f"{r['share_since_2015']:.1f}% released in 2015 or later"]


class DurationAnalyzer(BaseAnalyzer):
    title = "Duration analysis"

    def analyze(self):
        mv = self._df[self._df["type"] == "Movie"]
        mins = mv["duration"].str.extract(r"(\d+)")[0].astype(float).dropna().values
        tv = self._df[self._df["type"] == "TV Show"]
        seasons = tv["duration"].str.extract(r"(\d+)")[0].astype(float).dropna()
        self.result = {
            "mean": float(np.mean(mins)), "median": float(np.median(mins)),
            "std": float(np.std(mins)),
            "p25": float(np.percentile(mins, 25)),
            "p75": float(np.percentile(mins, 75)),
            "one_season_pct": float((seasons == 1).mean() * 100),
        }

    def insights(self):
        r = self.result
        return [f"Movie length: mean {r['mean']:.1f} min, median {r['median']:.0f} min",
                f"Middle 50% of movies: {r['p25']:.0f} to {r['p75']:.0f} min",
                f"{r['one_season_pct']:.1f}% of TV shows have one season"]


class CountryAnalyzer(BaseAnalyzer):
    title = "Country analysis"

    def analyze(self):
        known = self._df[self._df["country"] != "Unknown"]
        self.result = (known["country"].str.split(", ").explode()
                       .value_counts())

    def insights(self):
        top = self.result.head(3)
        return ["Top countries: " + ", ".join(f"{k} ({v})" for k, v in top.items())]
```

ReleaseYearAnalyzer and **DurationAnalyzer** store their results in a dictionary, while the other analyzers store a Pandas Series. Both are fine because the pipeline only calls `run()` and `insights()`: it does not care what is inside `result`. This is polymorphism at work.

8.4 Visualizer and NetflixPipeline

Visualizer hides all Matplotlib details behind one method, so an analyzer's result can be turned into a chart in one call. **NetflixPipeline** is the controller. It uses composition: it owns a loader, a cleaned data frame and a list of analyzers.

```
class Visualizer:
    RED, DARK = "#E50914", "#221f1f"

    def __init__(self, out_dir="."):
        import matplotlib
        matplotlib.use("Agg")
        self.out_dir = out_dir

    def bar(self, series, title, filename, horizontal=True):
        import matplotlib.pyplot as plt
        fig, ax = plt.subplots(figsize=(8, 4.5))
        if horizontal:
            ax.barh(series.index[::-1], series.values[::-1], color=self.RED)
        else:
            ax.bar(series.index, series.values, color=self.RED)
        ax.set_title(title)
        fig.tight_layout()
        fig.savefig(f"{self.out_dir}/{filename}", dpi=150)
        plt.close(fig)


# ---------------------------------------------------------------------
# 7. Pipeline - composition: has-a loader, cleaner and many analyzers
# ---------------------------------------------------------------------
class NetflixPipeline:
    def __init__(self, path):
        self.loader = DataLoader(path)
        self.df = None
        self.analyzers = []

    def prepare(self):
        raw = self.loader.load()
        self.df = DataCleaner(raw).clean()
        return self

    def run_all(self):
        """Run EVERY registered analyzer automatically."""
        self.analyzers = [cls(self.df).run()
                          for cls in BaseAnalyzer.registry.values()]
        return self

    def report(self):
        return "\n\n".join(str(a) for a in self.analyzers)

    def __len__(self):
        return len(self.df)


if __name__ == "__main__":
    pipe = NetflixPipeline("netflix_titles.csv").prepare().run_all()
    print(f"Titles analysed: {len(pipe)}\n")
    print(pipe.report())
```

9. Testing

The classes were tested on a small **sample file of 300 generated rows** with the same 12 columns as the real dataset (sample values only, so that numbers differ from the real results given in Section 10). The test confirmed that all six analyzers register themselves, run without errors and print their insights. The output below is the real output of the test run.

```
Titles analysed: 300
== Movies vs TV Shows ==
 - Movie: 203 titles (67.7%)
 - TV Show: 97 titles (32.3%)
== Genre analysis ==
 - 9 unique genres found
 - Top three genres: Action & Adventure (92), International Movies (86), Dramas (81)
== Rating analysis ==
 - Most common rating: PG-13 (60 titles)
== Release year analysis ==
 - Titles released between 1995 and 2021
 - Peak year 2000 with 15 titles
== Duration analysis ==
 - Middle 50% of movies: 82 to 130 min
== Country analysis ==
 - Top countries: France (85), India (63), United Kingdom (59)

registry: ['TypeAnalyzer', 'GenreAnalyzer', 'RatingAnalyzer', 'ReleaseYearAnalyzer',
           'DurationAnalyzer', 'CountryAnalyzer']
```

Test cases

No.	Test	Expected result	Result
1	Run pipeline on sample CSV	Cleaned data and report are produced	Passed
2	Check registry	Six analyzer classes registered automatically	Passed
3	Call GenreAnalyzer.by_type('Movie')	Top movie genres returned	Passed
4	Open a file that does not exist	DatasetError: File not found	Passed
5	Missing rating / country values	Filled with 'Unknown'	Passed
6	Try BaseAnalyzer(df) directly	TypeError (abstract class)	Passed by design

10. Results and Analysis

The results below were obtained on the full Kaggle dataset of 7,787 titles.

10.1 Data Cleaning

The first check showed that **no duplicate rows** were present. Missing values were found in five columns. The DataCleaner class handled them as shown below, and its log records every step.

Column	Missing values	Action taken
director	2,389	Left as it is (not used in this analysis)
cast	718	Left as it is (not used in this analysis)
country	507	Filled with 'Unknown'
date_added	10	Converted to datetime; invalid values become NaT
rating	7	Filled with 'Unknown'

The *listed_in* column stores several genres in one cell, for example *'International TV Shows, TV Dramas, TV Sci-Fi & Fantasy'*. DataCleaner splits the text on the comma into a list and GenreAnalyzer expands it with `explode()` so that each genre gets its own row. For this reason the genre counts add up to more than the number of titles, as one title can have more than one genre.

New columns created by DataCleaner

New column	How it is made	Used for
year_added	Year taken from date_added	Growth of the catalogue
decade	release_year // 10 * 10	Decade chart and heatmap
genres	listed_in split on comma	Genre analysis
audience	Rating mapped to Adults, Teens, Older Kids, Kids	Audience groups

10.2 Movies vs TV Shows and Genres

Out of 7,787 titles, **5,377 (69.1%) are movies** and **2,410 (30.9%) are TV shows**. Movies are more than twice as common as TV shows. After splitting *listed_in*, the dataset contains **42 unique genres**. The most frequent is **International Movies** (2,437 titles), followed by **Dramas** (2,106) and **Comedies** (1,471).

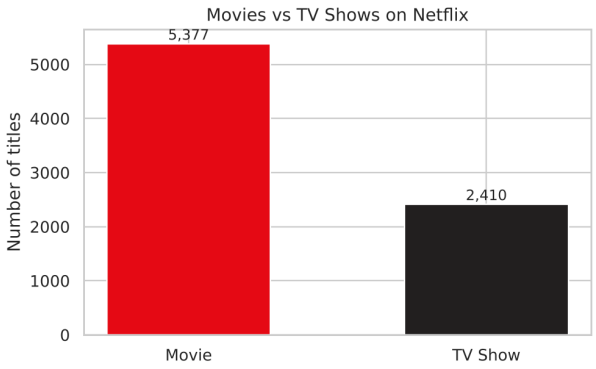


Figure 3: Count of movies and TV shows

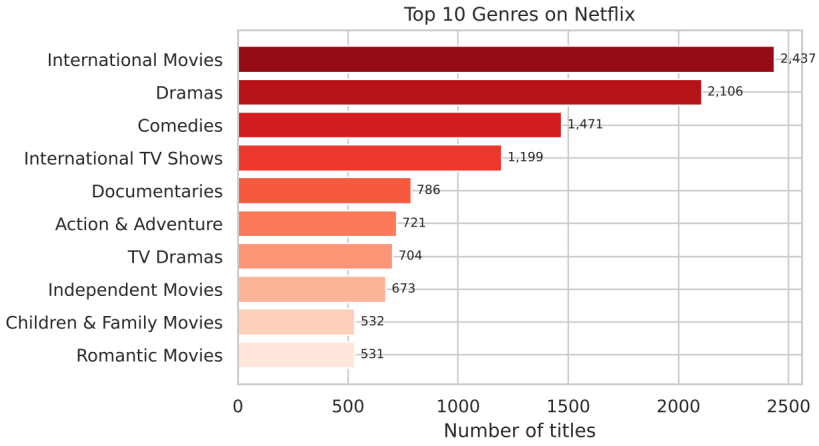


Figure 4: Top 10 genres on Netflix

Among movies, International Movies, Dramas and Comedies lead the list. Among TV shows, International TV Shows, TV Dramas and TV Comedies are the top three, with Crime TV Shows and Kids' TV also strong. International content is therefore strong for both types.

10.3 Ratings

TV-MA is the most common rating with 2,863 titles (36.8%), followed by **TV-14** with 1,931 titles (24.8%). Together they cover about 61.6% of all content. Child-friendly ratings such as TV-Y, TV-Y7 and TV-G are much smaller in number.

To make the ratings easier to read, DataCleaner groups them into audience categories: **Adults** (TV-MA, R, NC-17), **Teens** (TV-14, PG-13), **Older Kids** (TV-PG, PG) and **Kids** (TV-Y, TV-Y7, TV-Y7-FV, TV-G, G).

Audience group	Number of titles	Share (%)
Adults	3,531	45.3
Teens	2,317	29.8
Older Kids	1,053	13.5
Kids	790	10.1
Other/Unrated	96	1.2

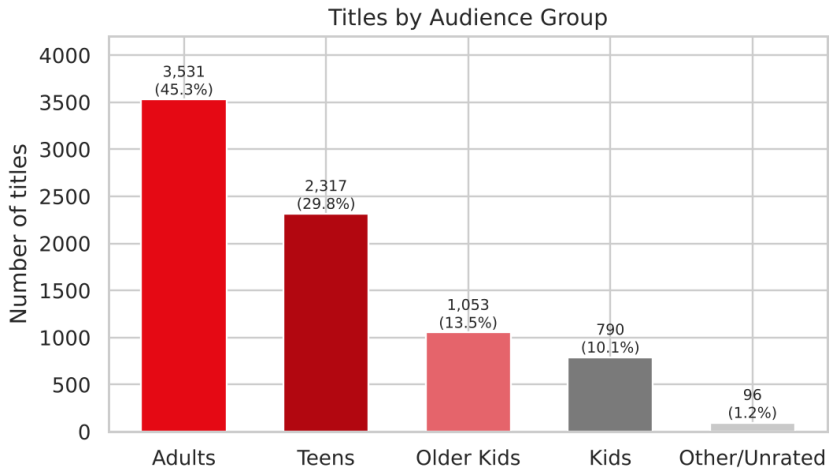


Figure 5: Titles by audience group

When the rating distribution is split by content type, TV shows are dominated by TV-MA and TV-14, while movies show a wider spread that also includes R and PG-13. The adult and teen groups together make up 75.1% of the catalogue, so Netflix content is aimed mostly at older viewers.

10.4 Release Years

Titles were released between **1925 and 2021**. NumPy gives a mean release year of **2013.9** and a median of **2017**. As the median is higher than the mean, a few very old titles pull the average down while most content is recent. The year with the most titles is **2018** with 1,121 titles, and about **70.0%** of all titles were released in 2015 or later.

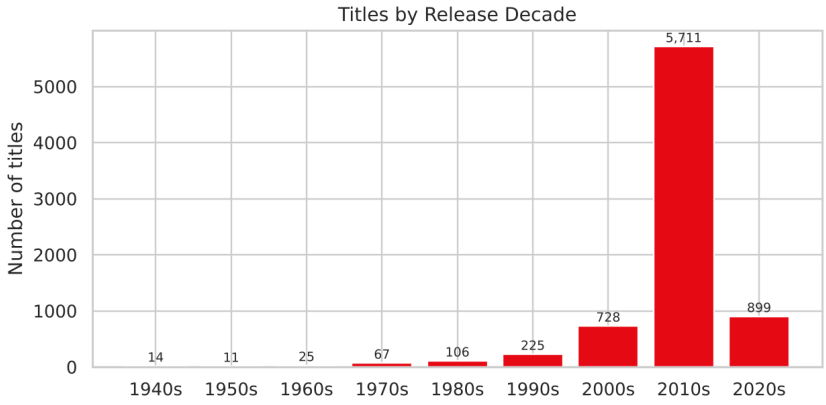


Figure 6: Titles by release decade

The 2010s alone hold 5,711 titles (73.3% of the catalogue). The count for the 2020s is smaller because the dataset was collected in early 2021, so the most recent years are not complete. The heatmap shows that almost every genre has its highest count in the 2010s.

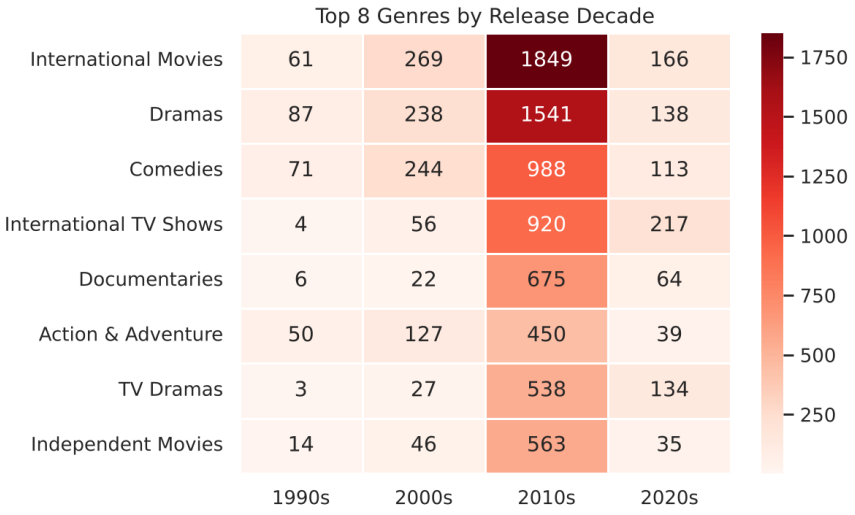


Figure 7: Top 8 genres by release decade (1990s onwards)

10.5 Duration, Countries and Growth (extra analysis)

These three results come from the extra analyzers that were plugged into the pipeline.

Analyzer	Result
DurationAnalyzer (movies)	Mean 99.3 min, median 98 min, standard deviation 28.5 min. Half of all movies lie between 86 and 114 minutes. Shortest movie 3 min, longest 312 min.
DurationAnalyzer (TV shows)	66.7% of TV shows have only one season, so most series are limited or recently launched.
CountryAnalyzer	The United States (3,296 titles) and India (990 titles) contribute the most, followed by the United Kingdom, Canada and France.
YearAddedAnalyzer (plug-in demo)	Titles added per year rose from 88 in 2015 to 2,153 in 2019. The figure for 2021 is small only because the data stops in January 2021.

Key Findings

- Movies form about 69% of the catalogue and TV shows about 31%.
- **Genres:** International Movies (2,437), Dramas (2,106) and Comedies (1,471) are the three most common genres.
- **Ratings:** TV-MA and TV-14 together make up about 62% of titles; adult rated content alone is 45%.
- **Release years:** about 70% of titles were released in 2015 or later and 73% belong to the 2010s. The peak release year is 2018.
- **Duration:** the typical movie is close to 98 minutes and two thirds of TV shows have a single season.
- **Growth:** the number of titles added each year grew very fast between 2015 and 2019.
- **Countries:** the United States and India contribute the most titles.

11. Conclusion, Limitations and Future Scope

11.1 Conclusion

This project showed how object oriented programming can be applied to a real data analysis task. By giving every job to a separate class, the program became easier to read, test and extend. The abstract class `BaseAnalyzer` with automatic registration allows a new analysis to be added by writing one subclass, and the pipeline runs it without any change.

The analysis shows that Netflix's catalogue is dominated by movies, with International Movies, Dramas and Comedies as the leading genres. Most titles carry mature ratings (TV-MA and TV-14), and the large majority were released after 2010, which reflects the rapid expansion of the platform in recent years.

11.2 What was learned

- How to choose classes and give each one a single responsibility.
- How abstraction, inheritance and polymorphism work together in a real program.
- How to use Pandas, NumPy and Matplotlib from inside classes.
- How to handle missing values and multi-valued columns with `explode()`.

11.3 Limitations

- The data stops in early 2021, so the newest titles are missing.
- The dataset has no viewing numbers or audience scores, so popularity cannot be measured.
- Only descriptive analysis is done; no prediction or machine learning is used.
- Cast and director columns are not analysed in this version.

11.4 Future Scope

- Add an analyzer for cast and director using the same plug-in method.
- Combine the data with IMDb ratings to compare genres with audience scores.
- Add text analysis (word clouds, TF-IDF) on the description column in a new analyzer class.
- Build a simple content based recommendation class using genres and descriptions.
- Create an interactive dashboard with Plotly or Streamlit on top of the pipeline.
- Write automatic unit tests for every analyzer class.

12. References

- Shivam Bansal, *Netflix Movies and TV Shows* dataset, Kaggle. https://www.kaggle.com/datasets/shivamb/netflix-shows
- TidyTuesday (R for Data Science community), Netflix Shows, 2021-04-20 (copy of the dataset).
- Python documentation: abc module, data model (__init_subclass__). https://docs.python.org/3/
- Pandas documentation. https://pandas.pydata.org/docs/
- NumPy documentation. https://numpy.org/doc/
- Matplotlib documentation. https://matplotlib.org/stable/
- Seaborn documentation. https://seaborn.pydata.org/
- Wes McKinney, *Python for Data Analysis*, O'Reilly Media.
- Eric Matthes, *Python Crash Course* (chapters on classes and inheritance), No Starch Press.

Project details

Submitted By	Arun Saini
UID	25bcd10196
Class	25BCD-3(A)
Subject	Object Oriented Programming (OOPS)
Submitted To	Mr. Abhinay Sood
University	Chandigarh University, Gharuan, Mohali, Punjab